

A Unified Benchmark for Privacy-preserving Vector Search [Experiment, Analysis & Benchmark]

Anne-Marie Kermarrec
EPFL
Lausanne, Switzerland

Mathis Randl
EPFL
Lausanne, Switzerland

Rafael Pires
EPFL
Lausanne, Switzerland

Martijn de Vos
EPFL
Lausanne, Switzerland

ABSTRACT

Vector search powers semantic search, recommendation systems, and retrieval-augmented generation (RAG). By design, the service answering a query sees both the query embedding and, usually, the corpus against which it is matched. This is a privacy breach for both the user issuing the query and the owner of the corpus. A family of cryptographic schemes (e.g., SAP, EMVP, BNTM, T_{IP}TOE) addresses that leak. However, as each scheme is published and evaluated on its own corpus, threat model, parameter choices, hardware, and metric conventions, the numbers cannot be compared directly. Consequently, a practitioner asking which one to deploy today has no defensible way to choose. We close that gap with a uniform experimental comparison, including a PLAINTEXT baseline and four cryptographic backends running over the same workload, hardware, and metric definitions. Under that ruler, the schemes spread across a Pareto frontier in privacy, performance, and recall rather than imposing a flat penalty on performance. We find that the performance of SAP matches PLAINTEXT, EMVP delivers cryptographic indistinguishability at a 4× throughput cost on CPU, BNTM adds malicious-server verifiability at a further 22× median-latency cost, and T_{IP}TOE hides the cluster choice itself, but incurs a 190× per-query cost compared to PLAINTEXT. GPU acceleration pays off for PLAINTEXT and SAP but not for EMVP or BNTM. All our experiment artifacts are publicly available for reproducibility.

PVLDB Reference Format:

Anne-Marie Kermarrec, Rafael Pires, Mathis Randl, and Martijn de Vos. A Unified Benchmark for Privacy-preserving Vector Search [Experiment, Analysis & Benchmark]. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/sacs-epfl/secure-vector-search>.

1 INTRODUCTION

Vector search is a key mechanism underlying modern information retrieval [20, 31]. It involves finding the vectors closest to a given

query vector among a collection that can hold billions of them [19, 39]. Each vector in this collection is typically a high-dimensional embedding representing a document, image, or other data item [38]. Vector search underlies recommendation systems [36] and code-search tools used daily by developers [11], as well as RAG pipelines, where a large language model (LLM) grounds its answers in documents fetched by nearest-neighbor vector search [22]. It is also an important component of commercial retrieval products and vector databases [30, 33] and is included in some agentic frameworks to manage the memory of LLM agents [47]. This makes vector search one of the most widely deployed pieces of infrastructure in retrieval systems nowadays.

Vector search is typically performed without any privacy protection: the service performing the vector search sees both the query embedding and the corpus embeddings against which it is matched. This exposes both the user issuing the query and the corpus owner to privacy violations. Research has shown that dense embeddings can be partially inverted to *recover substantial portions of the original text* (embedding inversion) [26, 41], even when the attacker can only query the embedding model as a black box (*i.e.*, without access to its internal weights). These inversion attacks allow the server to reconstruct parts of the corpus, and to detect which parts of the corpus are typically queried. This is problematic in high-stake domains such as healthcare, where a server observing query embeddings can learn which symptoms or conditions clinicians and patients are searching for, and even invert corpus embeddings to obtain the underlying patient records themselves [42].

The cryptographic community has answered this concern from substantively different directions. At the lightest end, SAP [13] scales and perturbs each vector with a small amount of noise before uploading it to the server, in such a way that the server can still compare distances between ciphertexts directly. However, this still enables the server to learn the geometry of the corpus, *i.e.*, which stored vectors lie close to which. EMVP [3] and BNTM [7] instead encrypt the vectors themselves and return inner products computed entirely under encryption, hiding both the corpus and the query from the server. BNTM additionally lets the client verify that the server’s answer is correct, a property the other schemes lack. T_{IP}TOE [15] goes a step further by layering private information retrieval over the entire pipeline so that the server does not even learn which part of the corpus a query targets. These schemes therefore span a wide spectrum, from lightweight perturbation to full protection of both the queries and the stored vectors. This spectrum

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

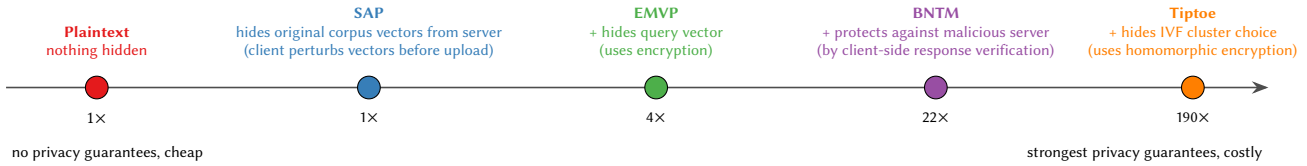


Figure 1: Comparing the five schemes on privacy guarantees and computational overhead. Each marker adds privacy guarantees over the one to its left, at an increase in per-query cost. The cost figures are the median end-to-end latency compared to the unencrypted baseline (PLAINTEXT); BNTM’s figure includes response verification. See Section 4 for the full experimental setup and results.

of privacy guarantees comes with widely different trade-offs in performance and retrieval accuracy. Figure 1 shows the performance cost of each privacy level.

Despite the different options for privacy-preserving vector search, no published study compares them on common ground. Each scheme is evaluated in its own paper, on its own corpus, with its own embedding model, hardware, and metric conventions that do not always refer to the same definition. As a result, a practitioner asking today which scheme to deploy has no defensible way to choose because no one has analyzed them under a common ruler.

This work presents a single, uniform comparison of privacy-preserving vector search schemes. We implement the baseline without any privacy schemes (PLAINTEXT) alongside all four privacy-preserving schemes in one common software harness, run every scheme over the same index, the same workload (the MS MARCO text corpus), and the same hardware, and measure all of them using the same metrics. Under a single benchmark, the differences between the schemes come out more clearly than any individual paper shows. The results debunk the belief, inherited from early fully-homomorphic prototypes, that encrypted vector search is unaffordable: even with corpus and query fully encrypted, EMVP answers 12 queries per second at recall 0.9 on the full MS MARCO corpus with 8.8 M passages, on one commodity server. The lightest-weight scheme (SAP) runs at practically the same speed as the unencrypted baseline at recall 0.9, on both CPU and graphics processing unit (GPU). Adopting SAP therefore costs almost nothing in speed, but it cannot guarantee privacy of the corpus itself, since the server can still tell which stored items are similar to one another. EMVP and BNTM fully encrypt the data but are considerably slower: EMVP is 4× slower than the PLAINTEXT baseline. BNTM, which adds the ability to detect a cheating server, sits further below that (22× slower than PLAINTEXT). The strongest scheme, TIPTOE, also hides which part of the corpus a query touches, but is a further 9× slower than BNTM, roughly two orders of magnitude below PLAINTEXT, on a corpus subset with 100 k passages. We also find that the amount of data sent over the network and the amount of data processed in memory rank these schemes differently: BNTM sends very little over the network but reads a large amount from memory internally, and it is this internal memory cost that predicts how well a scheme benefits from a GPU. We find that GPU acceleration helps PLAINTEXT and SAP, but not BNTM and EMVP.

This paper makes the following three contributions:

- We build a single evaluation harness for privacy-preserving vector search, which includes one corpus, one index configuration, one threat model, and one set of metric definitions that isolate the cryptographic primitive from the rest of the pipeline. This is what makes the four privacy-preserving schemes and the unencrypted baseline comparable to each other (Section 3).
- We benchmark all five schemes across recall, throughput, latency, and communication cost. We also measure the dimensions that decide whether a scheme is practical: how each scheme performs on CPU versus GPU, how its throughput scales across CPU cores, and how expensive it is to build the index from scratch (Section 4).
- We release all results underlying every experiment, so the comparison is fully transparent and reproducible.

2 BACKGROUND

We first outline how vector search operates, and then introduce our threat model. We then discuss the four state-of-the-art schemes for privacy-preserving vector search we include in our benchmark, which span a wide range of privacy guarantees, and an unencrypted baseline.

2.1 Vector search and inverted file (IVF)

Vector search. A vector-search service stores a corpus of N vectors with dimensionality d . These vectors are typically embeddings that are generated from documents, e.g., using embedding models such as sentence transformers [38] for textual data or image encoders for visual data [35]. Given a query vector q issued by a client, the service returns the k items most similar to q , a process known as nearest neighbour search (NNS) [28]. Typical similarity distances used in vector search include L2 (Euclidean) distance, inner product (dot product), and cosine similarity. Contemporary vector databases can contain billions of vectors, which makes an exact search, e.g., a brute-force query that scans all N vectors with $O(Nd)$ compute cost per query, prohibitively slow. Therefore, the majority of vector-search services rely on approximate NNS where retrieval is approximate [17]. With approximate NNS, an index built over all vectors narrows the search to a subset of candidates that, with high probability, contains the true nearest neighbors.

Vector search using IVF. Modern unencrypted retrieval increasingly relies on graph indexes such as HNSW [24] and DiskANN [18], which are often the faster choice on central processing unit (CPU).

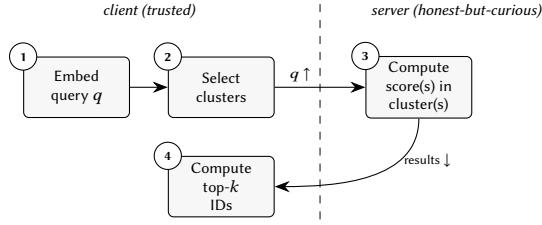


Figure 2: The four steps performed by the client and server in our per-query pipeline, shared by all five schemes, for privacy-preserving vector search using IVF.

These indexes traverse the data along a query-dependent path, inspecting each candidate to decide which edge to follow next. That data-dependent inspection is exactly what end-to-end encryption must hide. As a result, the three schemes that encrypt the vectors end to end (TIPTOE, EMVP, and BNTM) cannot use graph indexes at all. Since our unencrypted baseline and SAP could use them, but the other three schemes cannot, using graph indexes would complicate the comparison: any speed difference would mix the index choice with the cryptographic primitive used.

We therefore run all five schemes over a shared IVF index [19, 40] instead, which is a commonly-used index family that all five schemes can natively support. An IVF index works as follows. An offline k -means algorithm first partitions the vectors into roughly $\sqrt{N}$ clusters. At query time, the cluster centroids are sorted by distance to q , the closest n probe of them are picked, and the search analyzes only the distance to vectors inside those clusters. The n probe parameter trades off recall with computational cost, since larger values touch more clusters and recover more true neighbors at the cost of performing additional distance comparisons. The IVF per-cluster scan is embarrassingly parallel and therefore maps cleanly onto hardware accelerators such as GPUs.

In this work, we assume a client holding a corpus. This client first generates corpus embeddings and then uploads these embeddings to a server, which will perform the vector search on these corpus embeddings. Figure 2 sketches the pipeline when a client issues a query q to the server, which consists of four steps: (1) the client *embeds* its query q using the same embedding model as used when creating the corpus embeddings, (2) a *routing* step picks the n probe target clusters, (3) the server *scores* the candidates inside those clusters, and (4) the top- k IDs are returned to the client. The routing decision, *i.e.*, computing distances to centroids and picking the n probe closest clusters to analyze, is performed by the client, who holds the plaintext query and cluster centroids. We remark that all schemes, except TIPTOE, reveal to the server which clusters a query targets. Thus, the server learns n probe cluster indices for every query even though it never sees the query vector itself when using EMVP and BNTM. The scoring step is where the schemes diverge most in what they leak to the server, ranging from the query, corpus, and scores in plaintext down to nothing beyond the routed cluster index.

2.2 Threat model

By default, we assume an honest-but-curious server that faithfully executes the vector search mechanism but exploits its observations to infer as much private information as it can. It is persistent: it retains every query it observes, along with the relative timing between queries, indefinitely. The server stores the corpus vectors and associated IVF indexes. The client, however, is fully trusted: it does not leak its own queries, and it safeguards any secret key material the scheme issues to it (*e.g.*, the symmetric key used by SAP). We note that BNTM also defends against a server that deviates from the protocol by returning erroneous responses; we further discuss this property in Section 2.3.

The four cryptographic schemes differ in whether their privacy guarantee, *i.e.*, bounding what the server learns to some specified leakage, holds up under repeated observation. TIPTOE, EMVP, and BNTM hold up indefinitely: query after query, the server learns nothing beyond the system’s specified leakage. SAP does not: even a single view of the perturbed corpus reveals the relative distances between points. Repeated queries make this worse: each query is perturbed independently, so a server observing many queries toward the same region of the embedding space can average away the noise and have precise insight into the access patterns of the users. However, noise-based schemes like SAP provide mitigation against inversion attacks [42], which makes them useful in production when only the raw contents of the documents are sensitive. The geometry still reveals which documents lie close to each other. A server that knows or guesses the embedding model can label these clusters with topics by matching them against embeddings of public text [14, 27].

2.3 The five schemes under comparison

We next discuss the five state-of-the-art schemes included in our comparison, which includes four privacy-preserving vector search mechanisms and a plaintext, unencrypted baseline. The four privacy-preserving schemes rely on two different kinds of security argument. SAP uses *symmetric encryption*: the client holds a secret key and applies a keyed scaling to each vector, so security reduces to keeping that key secret from the untrusted server; unlike EMVP and BNTM, the resulting ciphertexts are not indistinguishable from random. The other three privacy-preserving schemes (EMVP, BNTM and TIPTOE) instead rely on a *computational hardness assumption*: each approach is secure only so long as a particular math problem stays infeasible to solve, which no one has proven. Those problems belong to the *learning* family, which all state that recovering a hidden linear function from noisy observations is computationally infeasible. They differ in the algebraic setting and in how long the assumption has been studied: EMVP relies on *secret dual codes*, BNTM on learning parity with noise (LPN) [4], and TIPTOE on learning with errors (LWE) [37]. We now outline each of these schemes, and also visualize the steps between the client and server within each privacy-preserving scheme in Figure 3. Table 1 also summarizes the five schemes and their privacy guarantees.

PLAINTEXT. This is the baseline and default setting. Vectors are stored and searched in the clear, without cryptographic protection.

Table 1: The five schemes considered in this benchmark and their privacy guarantees. *Security basis*: whether confidentiality reduces to keeping a key secret or to the hardness of a computational problem. *Assumption*: the specific hardness assumption, where applicable. *Leakage bounded under repeated queries*: whether the server learns nothing beyond its specified leakage even after repeated queries. *Hides IVF access pattern*: whether the server cannot tell which IVF cluster a query targets. *Malicious-server verifiability*: whether the client can verify that returned results match the requested computation. *Graph-index compatible*: whether the scheme is compatible with graph-based approximate nearest neighbour (ANN) indexes.

	PLAINTEXT	SAP	EMVP	BNTM	TIPTOE
Security basis	–	key secrecy	computational hardness		
Assumption	–	–	dual codes	LPN	LWE
Leakage bounded under repeated queries	✗	✗	✓	✓	✓
Hides IVF access pattern	✗	✗	✗	✗	✓
Malicious-server verifiability	✗	✗	✗	✓	✗
Compatible with graph-based index	✓	✓	✗	✗	✗

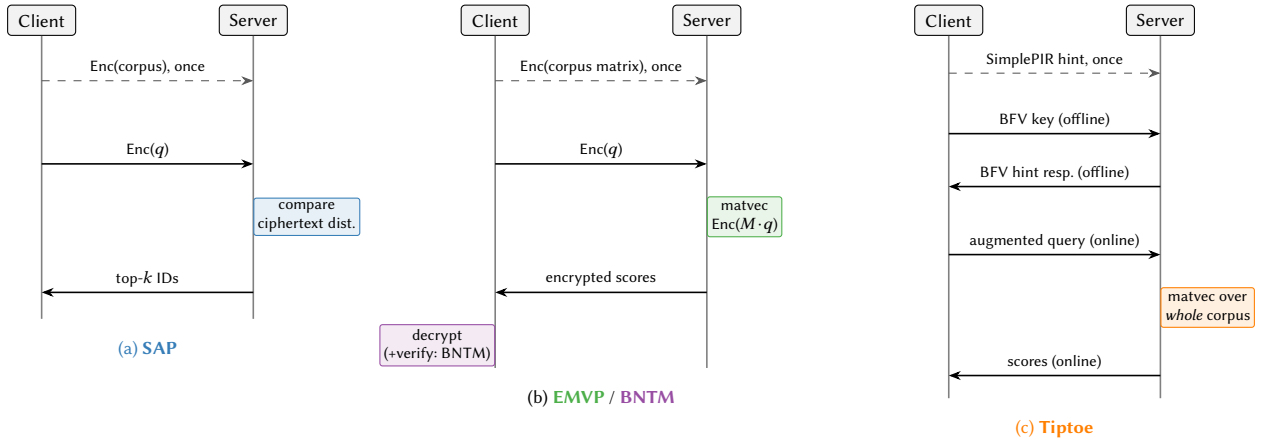


Figure 3: Communication between the client and server, and the performed operations during each of the four privacy-preserving vector search mechanisms, with time running from top to bottom. Dashed arrows indicate one-time setup operations and solid arrows indicate operations performed for each query.

SAP [13]. The main idea of SAP is to use a distance-comparison-preserving encryption (DCPE) scheme that lets the server compare distances between encrypted vectors directly, without ever decrypting them, also see Figure 3(a). The client holds a secret key and applies the same encryption to every vector it handles: it scales the vector by a per-key factor and adds a small, freshly sampled random perturbation. It encrypts the corpus once, at index-build time, and uploads only the ciphertexts to the server. At search time, the client encrypts each query the same way. We note that scaling the vectors alone would be exactly distance-preserving but insecure, since an attacker could perfectly recover the corpus geometry. The perturbed ciphertexts still approximately preserve the relative distances between points, up to controlled noise, the server can compare ciphertexts directly. It routes IVF queries and scans candidate clusters using the same code paths it would use on plaintext vectors, without ever decrypting anything. A privacy parameter β controls the magnitude of the perturbation added to each vector: a larger value of β blurs the distance order more aggressively, yielding better privacy but lower recall. Smaller values of β approach the recall of the plaintext baseline (also see Section 4.5).

EMVP [3]. This scheme has been designed for settings with an honest-but-curious server threat model and hides both the query and the corpus contents from the server by computing entirely under encryption, also see Figure 3(b). The client encrypts the corpus once, as a matrix of ciphertexts, and uploads it to the server. IVF cluster routing still happens in the clear, so the query only needs to be encrypted with respect to the routed cluster’s block. At query time, the client sends an encrypted query vector, and the server answers by computing a *matvec*, an encrypted matrix-vector product between the routed cluster’s ciphertext matrix and the encrypted query, entirely under encryption; the result is returned to the client, who decrypts it to obtain the cluster’s inner-product scores. EMVP comes with the strong guarantee that to the server, the ciphertexts are indistinguishable from random, so it learns neither the query vector, the corpus vectors, nor the inner products themselves, and this guarantee holds indefinitely under repeated observation. However, since cluster routing happens in the clear, the server does learn which cluster each query targets.

BNTM [7]. This scheme is, from a high-level perspective, comparable to EMVP since it also encrypts vectors into a matrix, uploads them to the server, and then computes the distances under encryption. There are two key differences. Firstly, BNTM relies on the learning parity with noise (LPN) assumption, which is a long-studied assumption that gives a more conservative security argument than the newer one EMVP uses (which is based on dual codes) [4]. Secondly, the client can verify, after receiving the encrypted response, that the server truthfully computed the requested inner products rather than fabricating them, with overwhelming probability and at a small per-query cost. This check uses *Freivalds’ algorithm*, which is a randomized algorithm used to verify that the result of a matrix multiplication is correct [12]. We quantify this verification cost in Section 4.5.

TIPTOE [15]. In contrast to the previous schemes discussed, TIPTOE also hides which cluster a query touches by having the server compute over the *entire* corpus for every query, rather than just the routed cluster, also see Figure 3(c). Cluster routing itself stays client-side: the client scans the public centroids locally to pick its target cluster, and that choice never leaves the client. It then builds an *augmented query* that spans all clusters, carrying the query vector in the target cluster’s block and zeros everywhere else, and encrypts the whole vector under LWE [37]. The server answers it with SIMPLEPIR [16], a single-server private information retrieval (PIR) scheme that replies to an encrypted query by multiplying it against the entire database, *i.e.*, a single matvec of the *whole* corpus matrix against the ciphertext. Its per-query work is therefore proportional to the whole corpus rather than to the target cluster. The client also needs a SIMPLEPIR *hint* to decode the result; because the hint depends only on the corpus and not the query, this exchange (handled under Brakerski-Fan-Vercauteren (BFV) homomorphic encryption [10]) runs *offline*, ahead of the query, leaving only the *online* matvec on the critical path. Because the augmented query is encrypted, the server cannot tell which block is non-zero, so it learns nothing about which cluster was targeted; the decrypted result carries the inner-product scores for clusters chosen by the client alone. TIPTOE thus has the strongest privacy guarantees of all schemes by hiding both the vector contents and the cluster choice, but comes with very high per-query computational cost.

3 BENCHMARK SETUP

We next describe the setup of our benchmark, and provide the implementation details of our five included schemes.

Workload. As corpus, we generate vectors from the MS MARCO passage collection [29], which contains 8 841 823 passages. This dataset is frequently used to evaluate vector-based retrieval in RAG settings. We embed each passage with the E5-base-v2 embedding model [45] into one 768-dimensional vector. During our experiments, we consider two corpus sizes: a 100 k-passage subset, referred to as MS MARCO-100k, and the full 8.8 M-passage collection, used when the scale of the corpus would affect the target metric, referred to as MS MARCO-8.8M. We also deterministically sample 1 000 questions from the MS MARCO dataset and use these as query workload. By default, we report recall@10 , *i.e.*, the fraction of a query’s ten true nearest neighbors that appear among the

ten returned IDs: $|R_{10} \cap G_{10}|/10$, where R_{10} are the returned IDs and G_{10} the ground-truth neighbors. To obtain G_{10} , we perform a brute-force search using L2 distance in the unencrypted corpus. As similarity metric, we use L2 distance for PLAINTEXT and SAP, and inner product for EMVP, BNTM, TIPTOE. Since E5-base-v2 embeddings are unit-normalised, the top- k ranking induced by L2 distance is identical to the one induced by inner product, which lets us evaluate all schemes against a single ground truth despite their differing similarity metrics.

IVF configuration. Every scheme implements the same scoring interface and reads the same IVF partition for a given query. We set the number of clusters $n_{\text{centroids}}$ to $\lceil \sqrt{N} \rceil$, yielding 317 clusters for MS MARCO-100k and 2 967 clusters for MS MARCO-8.8M. To assign vectors to clusters, we run k -means with a fixed seed of 42 for 25 iterations of Lloyd’s algorithm [23], which is the standard k -means loop that alternates nearest-centroid assignment with centroid recomputation. The assignment of every vector to one of the $\lceil \sqrt{N} \rceil$ clusters, is computed once on the plaintext vectors and consistent across all five schemes. This yields a mean cluster size $m = N/c \approx 315$ for MS MARCO-100k and $m \approx 2 966$ for MS MARCO-8.8M. The schemes using cryptography encrypt the vectors only after they have been assigned to clusters, so encryption never changes the assignment. This makes a cross-scheme comparison clean: because the IVF index is identical for every scheme, any measured difference originates from the specifications of the underlying cryptographic scheme.

Per-scheme parameters. We configure each cryptographic scheme for 128-bit security (*i.e.*, $\lambda = 128$). For EMVP and TIPTOE, we pick parameters so that the best known attack on the hardness assumption the scheme rests on costs about 2^{128} operations, which is a standard cryptographic target and on the same level as AES-128. For BNTM, whose paper does not provide concrete LPN parameters, we instantiate $\lambda = 128$ through the parameter heuristic of the original paper; we could not certify the concrete security level of the resulting tuple. Among the parameter sets that meet these requirements, we take the lowest-overhead one. We provide all per-scheme parameters in the extended version of this paper.

Two different 128s appear in the paper and should not be confused. The first one is $\lambda = 128$ as discussed above: it bounds *privacy*, so breaking confidentiality costs about 2^{128} operations. This applies to EMVP, BNTM, and TIPTOE. The second one is a *soundness* bound specific to BNTM: its iterated-Freivalds verifier (used to verify the server response, see Section 2.3) runs $\lambda' = 3$ trials per query and therefore fails to catch a cheating server with probability at most 2^{-128} . SAP is the exception on both counts: it uses a 128-bit *symmetric key*, but its privacy against an honest-but-curious server comes from the distance-distorting perturbation β , not from a computational-hardness assumption. We evaluate different values of $\beta \in \{0, 0.25, 0.5, 0.75, 1\}$ and quantify the impact on recall in Section 4.5. Unless stated otherwise, we evaluate SAP with $\beta = 0$.

Implementation deviations. Our implementation differs from the original papers discussing the included schemes in two places. First, TIPTOE normally refines its clustering in two ways: it recursively splits oversized clusters to keep them balanced, and it assigns the 20 % of vectors nearest a cluster boundary to two clusters instead of

one. We turn off both features so that every scheme uses the same IVF partitions. We validated our TIPTOE implementation against the official Go implementation [15] (patched to match this change), and found that the two implementations give identical results, on both recall and the per-query top- k result IDs.

Second, BNTM has no public reference implementation. Thus, we follow the paper’s parameters [7] and implementation details to the best of our ability, but stop short of one refinement. The paper’s mask is *trapdoored*, meaning it carries a secret shortcut that lets the client undo it faster than a generic mask would allow. We implement that mask, and we decode through the shortcut it provides: the client removes the mask with one small dense product plus one sparse product, rather than with the full $O(mn)$ multiplication a generic mask would force. What we do not implement is the paper’s *recursive* refinement, which applies the same trick a second time inside the mask’s own dense factor. Our client unmasking therefore saves a constant factor over naive unmasking, where the full construction is sublinear in mn , so the BNTM client-side latency we report is still an upper bound. This is a performance caveat, not a privacy break: hiding the corpus still reduces to LPN, and the missing refinement leaves the server-side computation, the communication bytes, the recall, and the 2^{-128} malicious-server detection all unchanged. We also cannot certify the exact security level, since the paper does not provide the exact LPN parameters.

Hardware. We run all experiments on a single machine equipped with a dual-socket Intel Xeon Gold 6426Y, 32 physical cores plus simultaneous multithreading (SMT) for 64 logical cores and with 128 GB of system memory. This machine also contains an NVIDIA RTX 5000 Ada GPU (with 32 GB of video RAM) which we leverage for our experiments that use a GPU. The only thing that changes between a CPU and a GPU run is where the server-side scoring code runs (*i.e.*, computing the distances between the query and vectors in each cluster, step 3 in Figure 2).

Measurement. We apply a few standard controls so the timings are stable and reproducible. All experiments use the 64 logical cores by default; Section 4.6 shows that 32 threads would be slightly faster, a choice that affects all schemes alike. BNTM’s response verification is disabled by default, since a malicious server is outside our default threat model; we enable it only in the verification experiment of Section 4.5. We set every machine’s CPU frequency governor to performance, which holds the cores at their top clock instead of letting the frequency ramp up and down, so latency does not change with the processor’s power state. We also drop the operating system’s page cache between runs so each run starts cold and no scheme benefits from data another left in memory. When changing the number of threads in Section 4.6, we pin threads to socket 0 up to 16 threads and run larger counts unpinned: on our dual-socket machine the memory is split between the two sockets (a non-uniform memory access (NUMA) layout) and reaching the other socket’s memory is slower, so confining threads to one socket lets us attribute the efficiency drop in our parallel throughput experiments to real per-core contention rather than to stray cross-socket traffic. Finally, every figure uses the *realised* per-query communication cost, *i.e.*, the actual bytes that left or arrived for that query, not the per-cluster mean over the whole index; this distinction matters for clustered corpora whose cluster sizes vary

by an order of magnitude (we quantify this in Section 4.3). Unless stated otherwise, every experiment uses all 64 logical CPU cores within each query, and queries are issued sequentially; only the parallel-scaling sweep (Section 4.6) varies the thread count.

Reproducibility. All artifacts are made public¹ and every figure in this paper can be reconstructed from the raw measurements. We repeat every experiment three times and report mean values.

4 BENCHMARK RESULTS

Our benchmark answers the following main question: *at the operating points that matter to a vector-search deployment, what does cryptographic privacy cost in terms of computational and communication cost, and search accuracy?* We answer this through the following sub-questions:

- What is the trade-off between throughput and recall for all schemes included in the benchmark, on different corpus sizes and on a CPU and GPU (Section 4.1)?
- How much does the GPU speed up the different schemes in terms of throughput and query latency (Section 4.2)?
- What is the (per-query) online, offline and (one-time) setup communication cost incurred by each scheme (Section 4.3)?
- What is the end-to-end query latency of each scheme and how much do the different operations contribute to this latency (Section 4.4)?
- How does the perturbation strength β in SAP affect recall, and what is the compute overhead of response verification by the client in BNTM (Section 4.5)?
- How much does multi-threading affect per-query latency of each scheme (Section 4.6)?
- How long does it take for each scheme to build the index by the client (Section 4.7)?

4.1 The throughput-recall trade-off

Our first experiment measures the trade-off between throughput (the number of answered queries/s) and recall@10 for all five schemes and for MS MARCO-100k and MS MARCO-8.8M. For all schemes, except TIPTOE, we vary the *nprobe* parameter (*i.e.*, the number of clusters to check), which trades compute cost for recall. Since TIPTOE is architecturally different, we evaluate TIPTOE with two quantization levels: 3 and 4 bit. Since we are interested in the per-query cost, we measure throughput by issuing queries sequentially and then take the reciprocal of mean single-query latency. Here, we focus on the throughput-recall trade-off and later quantify the communication cost and end-to-end latency in Section 4.3 and Section 4.4, respectively.

Figure 4(a) shows the throughput-recall trade-off on CPU for MS MARCO-100k (all five schemes), and Figure 4(b) for MS MARCO-8.8M (all schemes except TIPTOE), both with a logarithmic horizontal axis. Figure 4(c) repeats the MS MARCO-8.8M measurements on the GPU and is discussed in Section 4.2. Figure 4(a) considers eight values of *nprobe*, powers of two from 1 to 128. The MS MARCO-8.8M runs in (b) and (c) sweep *nprobe* over 1, 8, 64 and 256. In both settings, increasing the *nprobe* parameter increases recall at the cost of throughput. Figure 4(a) shows that for the BNTM scheme,

¹See <https://github.com/sacs-epfl/secure-vector-search>.

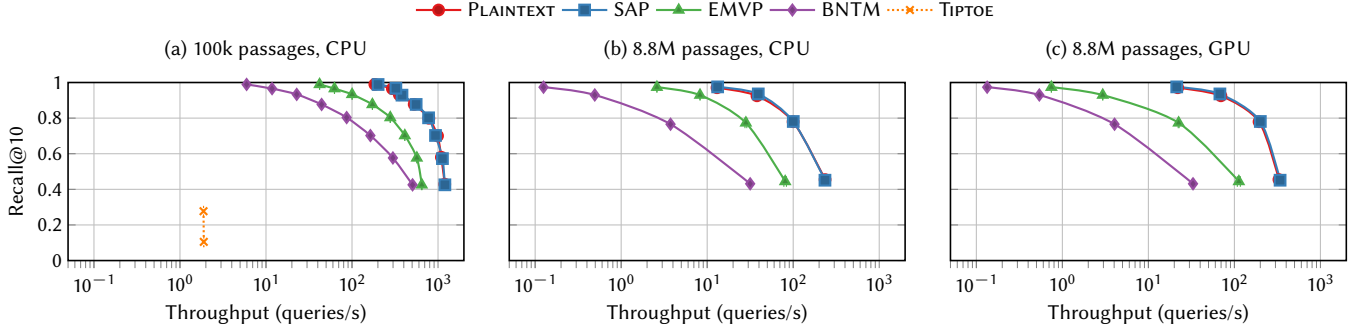


Figure 4: Recall@10 vs. sustained throughput (with a logarithmic horizontal axis). (a) MS MARCO-100k on CPU; (b) MS MARCO-8.8M on CPU; (c) MS MARCO-8.8M on GPU. TIPTOE is out of frame in (b) and (c), and runs at its own recall regime (≈ 0.11 – 0.28), so its curve sits low by construction.

for example, recall@10 increases from 0.43 for $nprobe = 1$ to 0.99 for $nprobe = 128$. At the same time, throughput for BNTM drops from 506 queries/s to 6.0 queries/s when increasing $nprobe$ from 1 to 128, an $84\times$ decrease. We observe similar trends for the other schemes, highlighting the recall-throughput trade-off.

We also observe in Figure 4 that the schemes separate into distinct tiers whose gaps widen as the privacy guarantee strengthens. The recall and throughput of SAP is very close to that of PLAINTEXT across every operating point on every chart. In other words, the privacy offered by SAP essentially adds no compute overhead because the server side runs the same scoring computation as PLAINTEXT but on shuffled ciphertexts. The throughput performance of EMVP at a recall of ≈ 0.9 is $3.5\times$ below that of PLAINTEXT with MS MARCO-100k and $4\times$ below with MS MARCO-8.8M. The throughput of BNTM sits below EMVP at a recall of ≈ 0.9 : $4.4\times$ lower throughput (100k, recall@0.9) with MS MARCO-100k and $44.6\times$ lower throughput (8.8M, recall@0.9) with MS MARCO-8.8M. Growing the corpus from 100 k to 8.8 M passages preserves the ordering of the schemes (Figure 4(a) vs. Figure 4(b)).

The throughput of TIPTOE on the small corpus (1.9 queries/s) is far below all other schemes, two orders of magnitude below PLAINTEXT, and also comes at a much lower recall. TIPTOE is not plotted in Figure 4(b) and (c) because it is too slow at that scale: its per-query cost grows linearly in corpus size, which extrapolates the measured MS MARCO-100k latency to 46 s per query on MS MARCO-8.8M, resulting in a throughput of 0.02 queries/s. TIPTOE’s low recall is structural for both evaluated quantization levels and has two separate explanations. First, TIPTOE searches exactly one IVF cluster per query (the server never learns which), so any ground-truth passage that lies outside that cluster can never be returned; on our corpus only 44% of a query’s ground-truth top-10 passages fall in the cluster the query is routed to. Second, TIPTOE’s encrypted matvec operates on low-precision integers, so the embeddings must be quantised before scoring, which reorders passages within the one cluster that is searched. Together they leave recall@10 at 0.11 at 3-bit precision and 0.28 at 4-bit precision. These figures match the authors’ reference implementation within 0.1 percentage point (0.110 vs 0.111 at 3-bit), and we hold the clustering layer identical across all five schemes rather than apply TIPTOE’s published

recall-boosting extensions (recursive cluster splitting, boundary double-assignment), which would break the controlled comparison. TIPTOE therefore trades recall for sublinear communication, its declared design point; the Figure 4 position quantifies that trade at benchmark scale.

Takeaway. Cryptographic indistinguishability costs a small factor, not orders of magnitude: EMVP runs $4\times$ below PLAINTEXT at recall 0.9. SAP comes with almost no computational overhead but reveals the approximate corpus geometry; the throughput gap between SAP and EMVP is the price of hiding that geometry.

4.2 Speedup by the GPU

Next, we analyze how offloading the score computation to a GPU on the server affects the performance of all schemes, except TIPTOE. We consider a GPU-native implementation of TIPTOE beyond the scope of our benchmark.

Recall-throughput. Figure 4(c) shows the recall-throughput trade-off when using a GPU. Moving from CPU to GPU (Figure 4(b) vs. Figure 4(c)) helps them unevenly: at recall 0.9, PLAINTEXT and SAP gain throughput (48.7 to 91.2 and 53.4 to 98.5 queries/s, respectively), EMVP loses throughput (12.0 to 6.6 queries/s), and BNTM barely moves (also see Table 2).

This is because most of their per-query time is client-side work that the GPU is unable to speed up. Section 4.4 discusses this further by presenting a latency breakdown of server and client operations.

Effect on throughput. Table 2 compares CPU and GPU performance at the same recall of 0.9. The setup is comparable to the one used in Section 4.1. Since no $nprobe$ setting lands on recall 0.9 exactly, the throughput values in Table 2 are obtained by taking, for each scheme, the two measured settings whose recalls lie just below and just above 0.9, and linearly interpolating the throughput between them. PLAINTEXT and SAP run 1.8 – $1.9\times$ faster on the GPU than on the 64-thread CPU baseline. EMVP loses throughput on the GPU at this corpus size because its encrypted matrix is memory-bandwidth-bound, not compute-bound: it performs a single modular multiply per 8 B ciphertext field element it reads, so

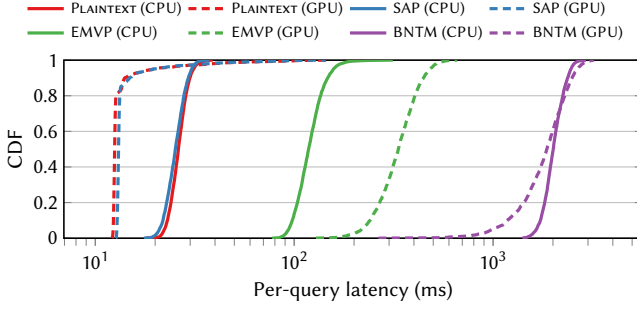


Figure 5: Per-query latency cumulative distribution function (CDF) for all schemes, on CPU and GPU, on MS MARCO-8.8M at $nprobe = 64$ (recall ≈ 0.93). This is a slightly higher operating point than the recall of exactly 0.9 in Table 2, so the throughput there is a bit higher than these latencies suggest.

Table 2: Throughput (in queries/s, or qps) at recall@10=0.9, on MS MARCO-8.8M. TIPTOE is omitted since it never reaches that recall.

Scheme	CPU (qps)	GPU (qps)	Speedup
PLAINTEXT	48.65	91.17	1.87×
SAP	53.40	98.50	1.84×
EMVP	12.01	6.63	0.55×
BNTM	1.09	1.18	1.08×

the GPU spends its time waiting on memory rather than computing. Ciphertext expansion makes this worse: the encrypted matrix is 91 GB for MS MARCO-8.8M, which doesn’t fit in the 32 GB of GPU memory. Thus, the encrypted matrix cannot stay resident and streams cluster-by-cluster over the PCIe bus, which is much slower than the on-device memory the PLAINTEXT and SAP score computation logic read from. BNTM sits at single-digit throughput at recall 0.9 (1.09 qps on CPU and 1.18 qps on GPU): its per-query time is dominated by client-side decoding (see Section 4.4), which the GPU does not speed up. Figure 5 complements Table 2 and shows the distribution of per-query latency at recall ≈ 0.9 .

The PLAINTEXT and SAP GPU curves sit left of their CPU counterparts along the whole distribution, EMVP’s sits right along the whole distribution, and BNTM’s two curves overlay around 2 s. The GPU runs do spread more (p95 at 1.5× the median, vs 1.2× on the CPU), but even at p95 PLAINTEXT and SAP stay 1.6× faster on the GPU. We attribute the wider spread to per-query kernel-launch and host-device synchronization jitter, which is large relative to the short GPU kernels.

Takeaway. Offloading the score computation to the GPU on the server pays off only where this computation is not memory-bound: PLAINTEXT and SAP gain 1.8–1.9× in throughput, but EMVP and BNTM show little increase, or even performance degradation.

4.3 Communication cost

Next, we quantify the communication cost of each scheme in our benchmark. We particularly focus on the communication volume between the client and server, and distinguish between four types of traffic. First, we measure the *one-time setup cost* that encrypts and uploads the corpus at index time to the server, which is a cost that is amortised across every subsequent query. Second, we measure *online per-query traffic* (up- and download volume), e.g., the client uploading the query embedding and the server replying with encrypted scores. Third, for TIPTOE we consider a per-query *offline* round-trip that precedes the online MATVEC computation, in which the client during the offline phase sends a BFV-encrypted secret key to the server and receives the BFV-encrypted hint response from the server. Fourth, we measure the volume of data that the server is reading from the memory per scoring call, which predicts the memory bandwidth requirement by the server. We run the communication cost experiments on both MS MARCO-100k and MS MARCO-8.8M, while ensuring that all schemes, except TIPTOE, achieve comparable recall@10 ≈ 0.9 by fixing $nprobe = 32$ and $nprobe = 64$, respectively. TIPTOE runs at its own recall@10 ≈ 0.28 point (similar to Section 4.1) but its recall does not influence the communication cost in our setup because this scheme always queries a single cluster.

Table 3 shows the four types of communication costs for each of the five schemes, for both MS MARCO-100k and MS MARCO-8.8M. The schemes fall into three communication shapes: PLAINTEXT pays only the online round-trip; SAP, EMVP and BNTM add the one-time setup upload; TIPTOE additionally pays a per-query offline round-trip before the matvec computation.

Setup cost. The setup column is where the privacy-preserving IVF schemes’ upfront tax lands. SAP uploads the DCPE-encrypted corpus, with a size of 614 MB for MS MARCO-100k; this is the cheapest compared to EMVP and BNTM because its ciphertexts keep the plaintext dimension d . EMVP and BNTM both upload an $m \times n$ encrypted matrix per cluster over 317 clusters ($n > d$), resulting in a setup size of 1.0 GB and 819 MB, respectively. These costs are one-time and amortise across subsequent queries. PLAINTEXT has no setup cost because no privacy constraint forces the corpus through the client: the server can ingest it directly. If the corpus instead starts at the client, PLAINTEXT would upload the raw vectors, 307 MB for MS MARCO-100k: half of SAP’s setup cost, because SAP ciphertexts use 8 B per coordinate instead of 4 B. TIPTOE’s setup cost comprises the 16 MB SimplePIR hint and is almost two orders of magnitude smaller than the setup cost for EMVP because the hint is $m_{\max} \times n_{\text{LWE}} \times 8 \text{ B}$ and $n_{\text{LWE}} = 2048$ is much smaller than $m \times n$ for the IVF encrypted matrix used by EMVP and BNTM. On MS MARCO-8.8M, the setup uploads of SAP, EMVP and BNTM grow linearly with N (88×) to 54 GB, 91 GB and 72 GB, while TIPTOE’s hint grows with the cluster size (9.4×) to 151 MB.

Per-query communication cost. On MS MARCO-100k, the per-query up- and download traffic varies significantly across schemes, between 3 KB and 1.9 MB, and 80 B and 7.2 MB, respectively. PLAINTEXT uploads a 3 KB query to the server (a vector with 768 coordinates at 4 B each) and SAP a 6 KB query (the same 768 coordinates at 8 B each); both return only the top- k identifiers and scores, which

Table 3: Per-query online and offline communication costs, and bytes read from memory by the server, on MS MARCO-100k (top, $n_{\text{probe}} = 32$) and MS MARCO-8.8M (bottom, $n_{\text{probe}} = 64$). All schemes, except TIPTOE, obtain a $\text{recall@10} \approx 0.9$; the communication cost of TIPTOE is independent of its recall. All rows are end-to-end measurements except TIPTOE[†] at 8.8M, whose costs are estimated from closed-form cost expressions (see the extended version of this work) since the LWE matvec computation cannot run at this scale.

		Comm. / query (online)		Offline / query	
Scheme	setup (one-time)	query $\uparrow$	response $\downarrow$	$\uparrow / \downarrow$	mem-read bytes / query
<i>MS MARCO-100k (nprobe = 32)</i>					
PLAINTEXT	—	3 KB	80 B	—	31 MB
SAP	614 MB	6 KB	80 B	—	31 MB
EMVP	1.0 GB	10 KB	7.2 MB	—	104 MB
BNTM	819 MB	8 KB	94 KB	—	83 MB
TIPTOE	16 MB	1.9 MB	7.8 KB	42 MB / 164 KB	16 MB
<i>MS MARCO-8.8M (nprobe = 64)</i>					
PLAINTEXT	—	3 KB	80 B	—	636 MB
SAP	54 GB	6 KB	80 B	—	636 MB
EMVP	91 GB	10 KB	126 MB	—	2.15 GB
BNTM	72 GB	8 KB	1.6 MB	—	1.69 GB
TIPTOE [†]	151 MB	18 MB	74 KB	42 MB / 492 KB	151 MB

accounts for 80 B at $k = 10$. EMVP and BNTM instead return one entry per corpus vector in every probed cluster, so their response grows with n_{probe} and with the cluster size, not with k . BNTM returns a single field element, 8 B, per corpus vector, with no secret-sharing layer: a probed cluster averages 368 vectors (IVF routing favours clusters denser than the corpus mean of $m \approx 315$), which requires communicating 2.9 KB to the client, and summing over the $n_{\text{probe}} = 32$ probed clusters gives 94 KB per query. EMVP returns a whole $s \times m$ block-product matrix per probed cluster (where $s = 76$ coded scalars per vector), which results in 219 KB download traffic per cluster and 7.2 MB once summed over the same 32 clusters. This is a $76\times$ penalty over BNTM at comparable recall, because the client must recover each score from s coded scalars instead of reading it directly. TIPTOE shows the opposite pattern: uploading the encrypted LWE query to the server incurs 1.9 MB (plus a 42 MB BFV offline-phase round-trip), while the LWE response size is only 7.8 KB. Comparing the top and lower part of Table 3 shows how these costs scale with corpus size. The query upload is unchanged for the four IVF schemes, since it depends only on the vector dimension, but grows $9.5\times$ for TIPTOE (1.9 MB to 18 MB), tracking $\sqrt{N}$. The responses of EMVP and BNTM grow $17\times$, following the mean cluster size ($9.4\times$) and the doubled n_{probe} , while PLAINTEXT and SAP still return only the 80 B top- k .

Memory traffic. Next, we analyze the per-query memory traffic required by the server to compute the scores, which is shown in the right-most column in Table 3. We measure this since the scoring computation on the server has to potentially read many clusters from memory which can become a bottleneck as the corpus size increases. The score computation reads $O(m \times d_{\text{enc}})$ bytes from its own memory per cluster, where d_{enc} is the scheme’s effective row width (4-byte fp32 for PLAINTEXT and SAP, $n_{\text{EMVP}} = 1292$ 8-byte u64 for EMVP, $n_{\text{BNTM}} = 1024$ 8-byte u64 for BNTM). The encrypted schemes pay a $2.7\text{--}3.4\times$ memory-traffic premium for the same recall even when their wire-response is small (e.g., for BNTM), because

direct delivery shrinks the *returned scalars* but not the matrix the score computation must scan. TIPTOE’s matrix is the SimplePIR hint itself, which is 16 MB for MS MARCO-100k and 151 MB for MS MARCO-8.8M.

The differences in memory-bandwidth and network requirements matter when selecting a scheme for deployment: network cost determines what deployment costs on a constrained or metered link, whereas memory bandwidth is what bounds server throughput and what determines whether moving the scoring computation to a GPU pays off. Choosing a scheme based on a low network volume footprint alone therefore mispredicts throughput. BNTM is an example of this: it has the smallest response size of all cryptographic schemes (94 KB) but its in-memory scan cost is comparable to that of EMVP (83 MB vs 104 MB). The same gap pattern holds for MS MARCO-8.8M (Table 3): BNTM’s in-memory cost rises to 1.69 GB vs EMVP’s 2.15 GB.

The extended version of this paper derives closed-form expressions that yield the wire and in-memory byte counts for all schemes.

Takeaway. Based on network cost and memory bandwidth, the schemes rank differently: BNTM is the cheapest cryptographic scheme in terms of network volume yet reads as much memory as EMVP.

4.4 E2E latency and breakdown

Distribution of E2E latencies. Next, we quantify the per-query E2E latency for all schemes, whose distribution is shown in Figure 6(a). In line with previous experiments, all schemes, except TIPTOE, are evaluated at $\text{recall@10} \approx 0.9$ but TIPTOE operates at a lower recall ≈ 0.28 point. We measure the per-query latency of BNTM with server response verification disabled. Figure 6(a) shows that PLAINTEXT, SAP and EMVP have the lowest end-to-end latency: a median latency of 2.8 ms for PLAINTEXT and 2.6 ms for SAP, and

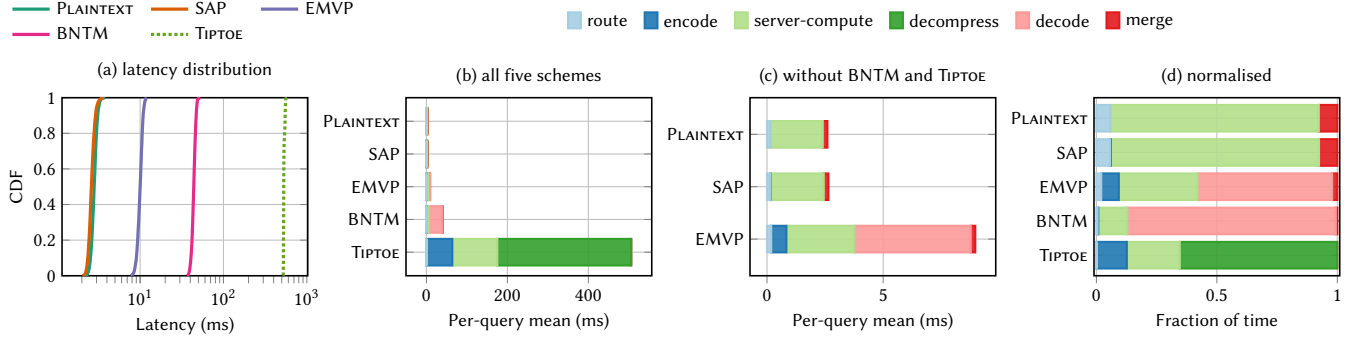


Figure 6: Per-query latency on MS MARCO-100k. (a) The distribution of E2E query latency for each of the five schemes; all schemes except TIPTOE are at the recall ≈ 0.9 operating point, TIPTOE at its own recall ≈ 0.28 point (Section 4.1). (b) The mean latency breakdown into the six operations for all five schemes; (c) the same without TIPTOE and BNTM, for readability; (d) the breakdown of (b) normalised to the total per-query time of each scheme.

10.0 ms for EMVP, with p99 below 12 ms. BNTM and TIPTOE have higher end-to-end latencies, with a median latency of 44 ms and 528 ms, respectively. We observe that TIPTOE’s CDF curve compared to those of the other scheme shows the lowest variance on the horizontal axis because its per-query budget is dominated by SimplePIR’s deterministic matvec computation, which runs in consistent time.

Latency breakdown. We further analyze query latency by recording for each scheme the time spent in each of the following six operations: *encode* (the client transforms or encrypts the query, step 1 in Figure 2), *route* (the client scores the query against the centroids and picks the nprobe clusters to probe, step 2 in Figure 2), *server-compute* (the server scores the probed clusters, step 3 in Figure 2), *decompress* (the client unpacks the server response), *decode* (the client recovers the scores from the response), and *merge* (the client combines the per-cluster candidates into the global top- k , step 4 in Figure 2). Figure 6(b) shows the mean duration of each operation for each scheme. For reference, Figure 6(d) shows the same results as in Figure 6(b) but normalizes each bar to 100%. TIPTOE shows the highest total per-query mean latency of 506 ms. This latency is dominated by the *decompress* operation (331 ms or 65% of the total latency), followed by the latency required for *server-compute* (111 ms, 22%) and *encode* (64 ms, 13%). TIPTOE’s per-query latency goes mostly to decompressing the SimplePIR response. BNTM has a total per-query mean of 43 ms, which is dominated by the client-side *decode* operation (35 ms, 81%). We run BNTM without verification here and analyze its verification cost in Section 4.5.

The total per-query mean latency of the PLAINTEXT, SAP and EMVP schemes are all under 9 ms, and for presentation clarity we show these latency breakdowns separately in Figure 6(c). PLAINTEXT and SAP both have a total per-query mean latency of 2.6 ms and their latencies are dominated by the *server-compute* operation (86%). EMVP has a total per-query mean latency of 9.0 ms and this latency is mainly attributed to the *decode* operation (56%).

From Figure 6 we conclude the following. PLAINTEXT and SAP are *server-compute* bound: 86% of their (small) compute time is spent in the scoring computation. The three end-to-end-encrypted

schemes are instead bound by *client-side response processing*: EMVP and BNTM by *decode* (dual-code decoding and trapdoor unmasking, respectively) and TIPTOE by decompressing the SimplePIR response.

Takeaway. Each scheme’s per-query cost is concentrated in a single pipeline segment: server-side scoring for PLAINTEXT and SAP, client-side decoding for EMVP and BNTM, and response decompression for TIPTOE. Optimisation has one clear target per scheme.

4.5 Varying the privacy knobs

SAP and BNTM expose configurable privacy parameters, and the two knobs trade against different axes. SAP’s perturbation strength β yields stronger snapshot privacy at the cost of recall, and BNTM’s verification toggle enables malicious-server detection at the cost of latency. We report the effect of both these parameters on recall (for SAP) and on latency (for BNTM) in Figure 7.

Perturbation β in SAP. Figure 7(a) shows recall@10 for SAP when sweeping the perturbation strength β from 0 (no perturbation) to 1, on the MS MARCO-8.8M dataset at nprobe = 64. Recall@10 decreases roughly linearly with β : from 0.93 at $\beta = 0$ to 0.82 at $\beta = 0.5$ (under which close vectors become indistinguishable) and 0.70 at $\beta = 1$; this drop is the price paid for symmetric snapshot privacy. At $\beta = 0$, recall closely matches the PLAINTEXT baseline at the same nprobe (the horizontal dashed line which sits at recall@10 = 0.925), as expected: the scale-only transform in SAP preserves the ranking. Note that SAP’s throughput-recall curve in the Figure 4 is computed with $\beta = 0$; the recall cost of perturbing vectors with $\beta > 0$ is the drop shown in Figure 7(a). β has no per-query latency cost: the perturbation is applied to the corpus at setup, so at a fixed nprobe the query-time work is identical for any β .

Toggling verification in BNTM. BNTM includes a verification mechanism that protects against a malicious response by the server, *i.e.*, the server cannot return a wrong answer on purpose without the client noticing, with detection probability $1 - 2^{-128}$ under the iterated-Freivalds verifier at $\lambda' = 3$ trials per query. Figure 7(b)

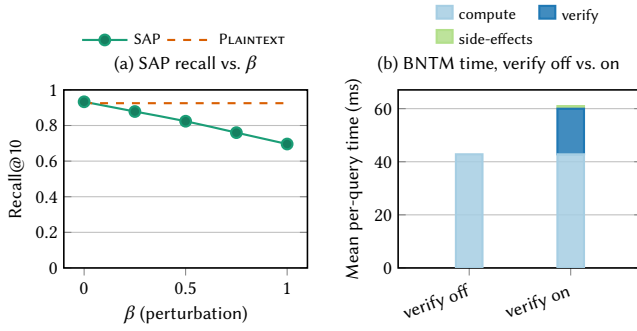


Figure 7: Privacy knobs in SAP and BNTM. (a) The effect of varying SAP’s perturbation strength β from 0 (no privacy) to 1 on recall@10, on MS MARCO-8.8M. For reference, the recall@10 of PLAINTEXT is 0.925 (dotted line). (b) BNTM per-query time breakdown at nprobe = 32, with and without verification enabled, on MS MARCO-100k.

shows the per-query time for BNTM on MS MARCO-100k with nprobe=32.

We split this time into three parts: *compute* (the full verify-off pipeline: the server computing scores in the cluster and client-side decoding), *verify* (running the Freivalds verifier), and *side-effects*, which captures the overhead the verify-on code path imposes on the rest of the pipeline. The verify segment takes 17 ms per query (40 % of the unverified 43 ms baseline); side-effects are negligible (1 ms). Thus, enabling malicious-server detection costs 1.4 $\times$ in per-query latency: 61 ms with verification vs. 43 ms without. The overhead is almost entirely the Freivalds checks themselves, whose trials stream the cluster matrix again. Randomly verifying queries can reduce the verification cost significantly, at the risk that a forged answer on an unchecked query goes undetected.

Takeaway. The two privacy knobs cost different things: SAP’s β trades off recall (0.93 to 0.70 for $\beta = 0$ and $\beta = 1$, respectively) for privacy and leaves latency unchanged. BNTM’s verification has a nontrivial latency (1.4 $\times$) cost, almost entirely due to Freivalds checks.

4.6 Multi-threading

Figure 8 shows how mean per-query latency (top) and parallel efficiency (bottom) changes as the CPU thread count N increases from 1 to 64, thus measuring how multi-threading speeds up each scheme. The sweep pins one thread per physical core of socket 0 up to 16 threads; larger counts run unpinned. The top panel shows that latency falls up to 8–16 threads inside socket 0, but by less than the thread count grows. Figure 8 (bottom) quantifies this as *parallel efficiency* $T(1)/(NT(N))$, where $T(N)$ is the mean per-query latency at N threads: 1 means ideal linear scaling, and adding threads without any latency reduction yields $1/N$. At $N = 16$ threads, parallel efficiency ranges from 0.68 (EMVP) down to 0.12 (BNTM). The second socket adds little: only EMVP gets faster (14 % from $N = 16$, pinned, to $N = 32$, unpinned), the other schemes

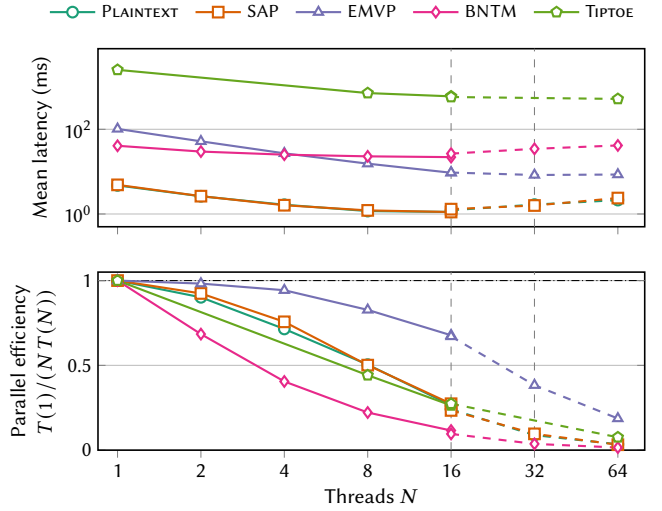


Figure 8: Performance of our schemes when increasing the number of threads, on MS MARCO-100k. Top: mean latency vs thread count, log-log. Bottom: speed-up over the single-thread baseline. Solid lines: NUMA-pinned to socket 0. Dashed lines: unpinned.

get slower because of cross-socket memory traffic. Using all 64 logical threads (SMT) increases latency for every scheme except TIPTOE, compared to 32 threads. With SMT, two threads share one physical core’s caches and memory ports; the score computation is already memory-bound, so the second thread adds contention without adding bandwidth. The shape is the same for all schemes except TIPTOE: this is evidence that the bottleneck is the host’s memory system, not the cryptographic primitive.

Takeaway. Shard per socket: only EMVP gains modestly from the second socket (14 %), and SMT adds contention rather than throughput.

4.7 Index build time

Finally, Figure 9 reports the (one-time) compute time it takes per scheme by the client to construct the index from the MS MARCO-100k corpus, *i.e.*, running the k -means clustering and, for the schemes using encryption, encrypt the corpus vectors. PLAINTEXT builds the full index in 10.5 s, which is essentially the k -means cost alone. SAP has a marginal compute overhead because of the vector scaling and perturbation. BNTM, EMVP and TIPTOE are slower (15.2 s, 21.8 s and 25.8 s, respectively), with all five schemes within a 2.5 $\times$ band. We attribute TIPTOE’s index build time to the SimplePIR hint matmul computation. All schemes share the 10.5 s clustering cost. The scheme-specific work on top of it ranges from negligible for SAP to 11.3 s for EMVP’s encryption and 15.3 s for TIPTOE’s hint computation, which match or exceed the clustering cost itself.

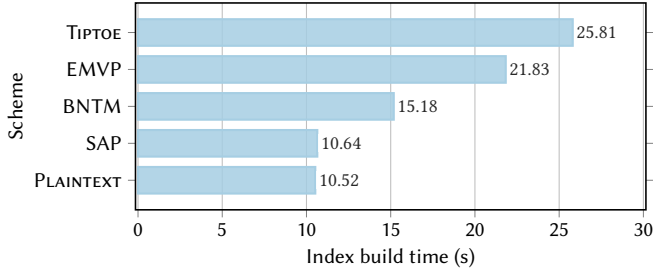


Figure 9: The time (in seconds) to build the index for each scheme from the MS MARCO-100k corpus.

Takeaway. Index build takes tens of seconds at 100k: all schemes share the 10.5 s k -means cost, and for EMVP and TIPTOE the encryption or hint computation costs about as much again.

5 LESSONS LEARNED AND ACTION POINTS

The key question for a deployment is which guarantee is worth which cost.

Action points. Our results translate into the following suggestions for practitioners working with vector search services:

- If the server may see the queries and corpus, use PLAINTEXT: every privacy scheme only adds cost.
- To hide the stored vectors at almost no performance cost, use SAP and pick the perturbation factor β to match the recall the application requires; accept that the server can still learn the approximate corpus geometry.
- If corpus vectors and the query must remain hidden, use EMVP: budget a $4\times$ throughput cost and $3\times$ the memory bandwidth per query, and do not plan on a GPU.
- If the server may also cheat, use BNTM and budget $1.4\times$ its own latency for client-side response verification.
- If the access pattern itself is sensitive, TIPTOE is the only option in this comparison but comes with $9\times$ more per-query latency compared to BNTM.

Implementation matters as much as protocol. A scheme’s measured cost depends as much on the implementation as on the protocol: the CPU-vs-GPU asymmetry, for example, tracks memory bytes read per query rather than anything in the cryptographic construction. We recommend that future evaluations hold the implementation substrate fixed and report protocol-level and implementation-level cost separately.

6 RELATED WORK

ANN benchmarking. ANN-Benchmarks [2] is the de-facto standard for comparing plaintext ANN systems on a common workload and metric, and the BIGANN challenge [39] extends it to billion-scale corpora. Neither covers cryptographic schemes. Our methodological contribution (Section 3) is to carry the common-harness idea to cryptographic schemes, so that the cost of privacy can be read on the same axes as the cost of recall.

Hardness assumptions. The four cryptographic schemes in this comparison rest on three distinct hardness assumptions. TIPTOE is built on LWE [37], the lattice assumption that underlies most modern fully-homomorphic encryption (FHE) and PIR constructions. BNTM is built on LPN [4], a long-studied assumption [32] whose concrete hardness is bounded by the BKW algorithm [5]. EMVP relies on a more recent assumption (secret dual codes, introduced in the original paper). The three occupy different points on the conservativeness-performance trade-off, which the per-query costs in Section 4 reflect.

Distance-comparison-preserving encryption. SAP [13] is the modern representative of a longer line of work on order-revealing and order-preserving encryption [6, 21, 34], which trades cryptographic indistinguishability for the server’s ability to operate directly on ciphertexts. SAP’s contribution is to extend that idea to approximate distance comparisons over real-valued vectors, as an IVF index needs.

PIR- and FHE-based private retrieval. TIPTOE’s server-side computation builds on SimplePIR [16], a low-overhead linearly homomorphic PIR scheme, and on BFV [10] for the per-query token. Earlier PIR systems such as XPIR [25] and SealPIR [1] reach similar privacy at substantially higher cost.

Encrypted matrix-vector products. EMVP [3] and BNTM [7] sit in a small but active line of work on encrypted linear algebra. A concurrent construction by Vaikuntanathan and Zamir [43] explores recursive trapdoored matrix variants for the same problem.

Encrypted ANN systems. TIPTOE [15] is the first end-to-end private vector-search service. Adjacent work on private record retrieval, including Splinter [44] and PIR-PSI [9], achieves related goals on structured key-value data rather than on dense vectors.

Trusted-hardware approaches. A different way to protect the vectors is to run the search inside a trusted execution environment, such as Intel SGX. The search then runs on plaintext inside the enclave at near-native cost, but the trust shifts from cryptographic hardness to the chip vendor’s attestation: every server in the deployment must support the chosen environment, and the enclave inherits the vendor’s side-channel track record. The direction has been explored for encrypted databases [46] and oblivious key-value stores [8], but we are not aware of a published TEE-based vector-search system measured on axes comparable to ours.

7 CONCLUSION

We presented a unified benchmark for privacy-preserving vector search that measured four cryptographic schemes (SAP, EMVP, BNTM, TIPTOE) against a PLAINTEXT baseline on a single shared testbed: one IVF index, one workload, one set of metric definitions, and the same hardware. Two findings hold beyond the individual schemes. First, wire bytes and memory bytes rank schemes differently: BNTM is cheap in network cost but expensive in memory access, and it is the memory requirements that predict the attainable speedup by using a GPU. Second, a scheme’s cost is as much an implementation property as a protocol one. These results debunk the claim that cryptographic privacy is unaffordable for vector search: it does not hold in our measurements on current hardware.

REFERENCES

- [1] Sebastian Angel, Hao Chen, Kim Laine, and Srinath Setty. 2018. PIR with Compressed Queries and Amortized Query Processing. In *2018 IEEE Symposium on Security and Privacy (SP)*. doi:10.1109/SP.2018.00062
- [2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020). doi:10.1016/j.is.2019.02.006
- [3] Fabrice Benhamouda, Caicai Chen, Shai Halevi, Yuval Ishai, Hugo Krawczyk, Tamer Mour, Tal Rabin, and Alon Rosen. 2025. Encrypted Matrix-Vector Products from Secret Dual Codes. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (Taipei, Taiwan) (CCS '25)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3719027.3765194
- [4] Avrim Blum, Merrick Furst, Michael Kearns, and Richard J. Lipton. 1994. Cryptographic Primitives Based on Hard Learning Problems. In *Advances in Cryptology — CRYPTO '93*. Springer Berlin Heidelberg, Berlin, Heidelberg. doi:10.1007/978-3-540-56617-8_1
- [5] Avrim Blum, Adam Kalai, and Hal Wasserman. 2003. Noise-tolerant learning, the parity problem, and the statistical query model. *J. ACM* 50, 4 (July 2003). doi:10.1145/792538.792543
- [6] Alexandra Boldyreva, Nathan Chenette, Younho Lee, and Adam O'Neill. 2009. Order-Preserving Symmetric Encryption. In *Advances in Cryptology - EURO-CRYPT 2009*. Springer Berlin Heidelberg, Berlin, Heidelberg. doi:10.1007/978-3-540-56617-8_1
- [7] Mark Braverman and Stephen Newman. 2026. Practical Secure Delegated Linear Algebra with Trapdoored Matrices. In *Theory of Cryptography*. Springer Nature Switzerland, Cham. arXiv:2502.13060
- [8] Emma Dauterman, Vivian Fang, Ioannis Demertzis, Natacha Crooks, and Raluca Ada Popa. 2021. Snoopy: Surpassing the Scalability Bottleneck of Oblivious Storage. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (Virtual Event, Germany) (SOSP '21)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3477132.3483562
- [9] Daniel Demmler, Peter Rindal, Mike Rosulek, and Ni Trieu. 2018. PIR-PSI: Scaling Private Contact Discovery. *Proc. Priv. Enhancing Technol.* 2018, 4 (2018). doi:10.1515/POPETS-2018-0037
- [10] Junfeng Fan and Frederik Vercauteren. 2012. Somewhat Practical Fully Homomorphic Encryption. *Cryptology ePrint Archive*, Paper 2012/144. <https://eprint.iacr.org/2012/144>
- [11] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. In *Findings of the association for computational linguistics: EMNLP 2020*. 1536–1547.
- [12] Rüdiger Freivalds. 1979. Fast probabilistic algorithms. In *International Symposium on Mathematical Foundations of Computer Science*. Springer, 57–69.
- [13] Georg Fuchsbauer, Riddhi Ghosal, Nathan Hauke, and Adam O'Neill. 2021. Approximate Distance-Comparison-Preserving Symmetric Encryption. *Cryptology ePrint Archive*, Paper 2021/1666. <https://eprint.iacr.org/2021/1666>
- [14] Paul Grubbs, Kevin Sekniqi, Vincent Bindschadler, Muhammad Naveed, and Thomas Ristenpart. 2017. Leakage-Abuse Attacks against Order-Revealing Encryption. In *2017 IEEE Symposium on Security and Privacy (SP)*. 655–672. doi:10.1109/SP.2017.44
- [15] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nikolai Zeldovich. 2023. Private Web Search with Tiptoe. In *Proceedings of the 29th Symposium on Operating Systems Principles (Koblenz, Germany) (SOSP '23)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3600006.3613134
- [16] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. 2023. One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA. <https://www.usenix.org/conference/usenixsecurity23/presentation/henzinger>
- [17] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 604–613.
- [18] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf
- [19] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011). doi:10.1109/TPAMI.2010.57
- [20] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*. 6769–6781.
- [21] Kevin Lewi and David J. Wu. 2016. Order-Revealing Encryption: New Constructions, Applications, and Lower Bounds. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (Vienna, Austria) (CCS '16)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/2976749.2978376
- [22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [23] S. Lloyd. 1982. Least squares quantization in PCM. *IEEE Transactions on Information Theory* 28, 2 (1982), 129–137. doi:10.1109/TIT.1982.1056489
- [24] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020). doi:10.1109/TPAMI.2018.2889473
- [25] Carlos Aguilar Melchor, Joris Barrier, Laurent Fousse, and Marc-Olivier Killijian. 2016. XPIR : Private Information Retrieval for Everyone. *Proc. Priv. Enhancing Technol.* 2016, 2 (2016). doi:10.1515/POPETS-2016-0010
- [26] John Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander Rush. 2023. Text Embeddings Reveal (Almost) As Much As Text. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore. doi:10.18653/v1/2023.emnlp-main.765
- [27] Muhammad Naveed, Seny Kamara, and Charles V. Wright. 2015. Inference Attacks on Property-Preserving Encrypted Databases. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS '15)*. Association for Computing Machinery, 644–655. doi:10.1145/2810103.2813651
- [28] Sameer A Nene and Shree K Nayar. 1997. A simple algorithm for nearest neighbor search in high dimensions. *IEEE Transactions on pattern analysis and machine intelligence* 19, 9 (1997), 989–1003.
- [29] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A Human Generated Machine Reading Comprehension Dataset. *CoRR* (2016). arXiv:1611.09268
- [30] OpenAI. 2026. File Search. <https://platform.openai.com/docs/assistants/tools/file-search> Accessed May 2026.
- [31] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *The VLDB Journal* 33, 5 (2024), 1591–1615.
- [32] Krzysztof Pietrzak. 2012. Cryptography from Learning Parity with Noise. In *SOFSEM 2012: Theory and Practice of Computer Science*. Springer Berlin Heidelberg, Berlin, Heidelberg. doi:10.1007/978-3-642-28110-3_1
- [33] Pinecone. 2026. Pinecone Vector Database. <https://www.pinecone.io/> Accessed May 2026.
- [34] Raluca Ada Popa, Frank H. Li, and Nikolai Zeldovich. 2013. An Ideal-Security Protocol for Order-Preserving Encoding. In *2013 IEEE Symposium on Security and Privacy*. doi:10.1109/SP.2013.38
- [35] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
- [36] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- [37] Oded Regev. 2009. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM* 56, 6, Article 34 (Sept. 2009). doi:10.1145/1568318.1568324
- [38] Nils Reimers, I. Sentence-BERT Gurevych, et al. 2019. Sentence embeddings using siamese BERT-networks. *arXiv preprint arXiv:1908.10084* (2019).
- [39] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2022. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track (Proceedings of Machine Learning Research, Vol. 176)*. PMLR. <https://proceedings.mlr.press/v176/simhadri22a.html>
- [40] Sivic and Zisserman. 2003. Video Google: a text retrieval approach to object matching in videos. In *Proceedings Ninth IEEE International Conference on Computer Vision*. doi:10.1109/ICCV.2003.1238663
- [41] Congzheng Song and Ananth Raghunathan. 2020. Information leakage in embedding models. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*. 377–390.
- [42] Yu-Che Tsai, Hsiang Hsiao, Kuan-Yu Chen, and Shou-De Lin. 2026. Concept-Aware Privacy Mechanisms for Defending Embedding Inversion Attacks. In *International Conference on Learning Representations (ICLR)*.
- [43] Vinod Vaikuntanathan and Or Zamir. 2025. Improving Algorithmic Efficiency using Cryptography. arXiv:2502.13065 [cs.CR]
- [44] Frank Wang, Catherine Yun, Shafi Goldwasser, Vinod Vaikuntanathan, and Matei Zaharia. 2017. Splinter: Practical Private Queries on Public Data. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. USENIX Association, Boston, MA. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/wang-frank>
- [45] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text Embeddings by Weakly-Supervised Contrastive Pre-training. *CoRR* (2022). arXiv:2212.03533

- [46] Wenting Zheng, Ankur Dave, Jethro G. Beekman, Raluca Ada Popa, Joseph E. Gonzalez, and Ion Stoica. 2017. Opaque: An Oblivious and Encrypted Distributed Analytics Platform. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. USENIX Association, Boston, MA. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/zheng>
- [47] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 19724–19731.

A APPENDIX

This appendix collects figures that supplement specific claims in the main body but did not fit there. Figure 10 extends the parallel-scaling reading to the full 8.8 M corpus; Figure 11 is a methodological sanity check on the shared IVF partition; and Figures 12 and 13 expand the CPU-vs-GPU reading from Section 4.2.

At the full 8.8 M corpus the parallel-scaling shape replays as at 100 k: the socket-boundary efficiency drop is the same for all schemes except TIPTOE. Figure 10 confirms that the shared-memory bottleneck named in Section 4.6 is corpus-invariant.

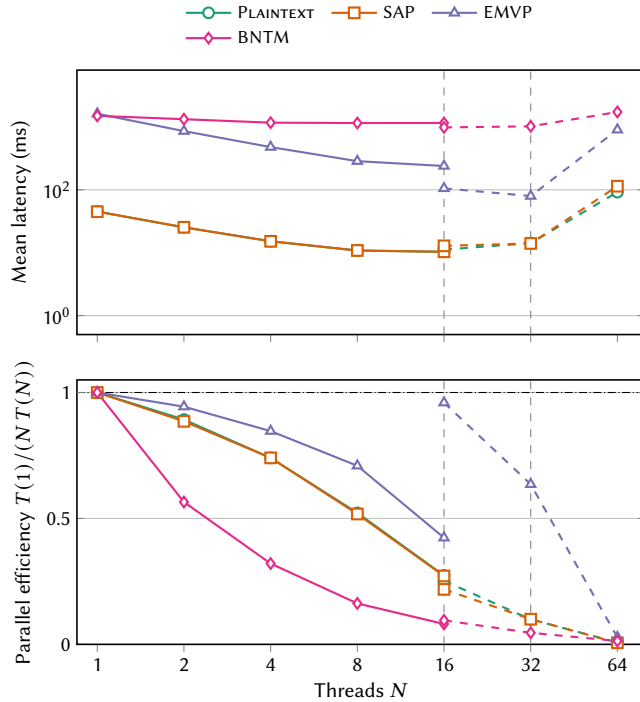


Figure 10: Parallel scaling on a dual-socket Xeon Gold 6426Y (32 physical cores + SMT), MS MARCO-8.8M, all schemes except TIPTOE. Same shape as the main-body PLAINTEXT-baseline figure but at the full corpus. TIPTOE is omitted from the 8.8 M scaling sweep because no TIPTOE-GPU build was measured and the CPU-only path is out of scope for cross-substrate scaling at this size.

Figure 11 is the methodological sanity check that lets the body attribute per-scheme cost differences to the cryptographic primitive rather than to the index structure. All schemes except TIPTOE overlay on the recall vs nprobe curve because every scheme rides

the same plaintext-trained partition, so the probed-cluster set is bit-identical across backends.

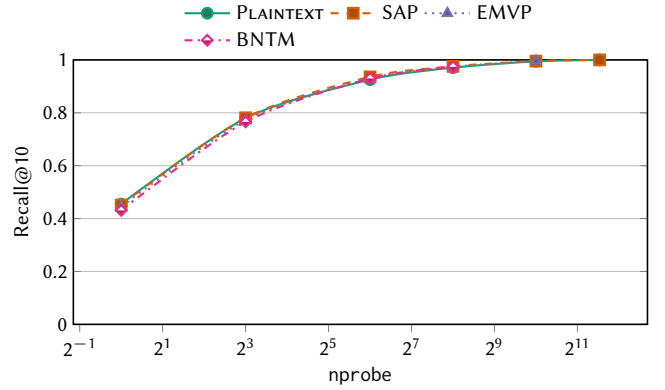


Figure 11: Recall@10 vs. nprobe for all schemes except TIPTOE on MS MARCO-8.8M. Reads off “how aggressive does the probe set need to be to reach X% recall” directly, instead of inferring from the latency / throughput Pareto in the main body. All four curves overlay tightly because every scheme shares the same plaintext-trained IVF partition. Source: MS MARCO-8.8M.

Figure 12 expands the Table 2 summary into per-(scheme, substrate) recall-throughput curves. PLAINTEXT and SAP shift left of their CPU baselines on the GPU; EMVP shifts right; BNTM overlays itself on both substrates. The three regimes named in Section 4.2 read off directly.

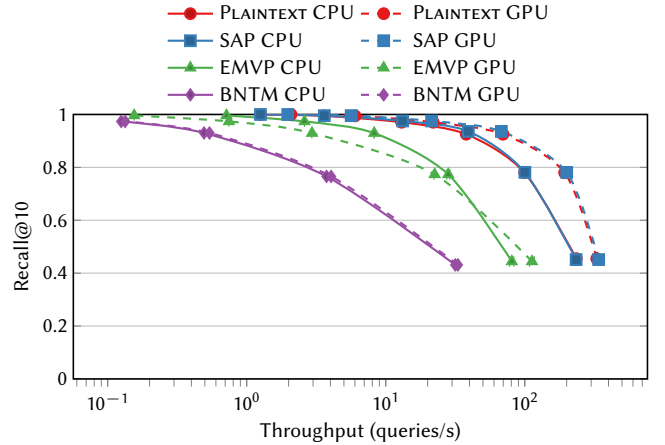


Figure 12: Recall@10 vs. sustained throughput, all schemes except TIPTOE, CPU solid vs GPU dashed of the same colour. PLAINTEXT and SAP translate cleanly to the GPU (1.8–1.9× over the 64-thread CPU baseline); EMVP loses on the GPU because its encrypted matrix–vector kernel is memory-bandwidth-bound and, at 91 GB, far exceeds device memory, so it streams over the PCIe bus rather than running resident; BNTM sits at single-digit qps on both sides. Source: Xeon Gold 6426Y (64 cores) + RTX 5000 Ada, MS MARCO-8.8M.

Table 4 isolates EMVP’s crossover: at $n_{\text{probe}} = 1$ a single small cluster fits on-device and the GPU wins, but as n_{probe} grows the per-query ciphertext volume climbs, the matrix no longer fits in device memory, and the CPU pulls progressively ahead, reaching $4.6\times$ at $n_{\text{probe}} = 1024$. Recall is identical across substrates (the kernels compute the same field matvec), so the gap is purely the memory-bandwidth effect of Section 4.2: the CPU feeds the matvec from dynamic random-access memory (DRAM) faster than the GPU streams it over the PCIe bus.

Table 4: EMVP single-query throughput (qps, mean of three repetitions) on CPU vs GPU at MS MARCO-8.8M, by n_{probe} ; recall@10 is identical on both substrates. The GPU leads only at $n_{\text{probe}} = 1$ (one small cluster, resident on-device); beyond that the encrypted matrix outgrows device memory and streams over the PCIe bus, so the CPU wins by a margin that widens with the per-query byte volume (CPU/GPU > 1 means CPU faster). Source: Xeon Gold 6426Y (64 cores) + RTX 5000 Ada, MS MARCO-8.8M.

n_{probe}	recall@10	CPU qps	GPU qps	CPU/GPU
1	0.444	80.7	112.7	0.72
8	0.773	28.2	22.5	1.25
64	0.929	8.26	2.95	2.80
256	0.974	2.60	0.75	3.49
1024	0.996	0.72	0.16	4.61

Figure 13 plots recall against the analytical eff-bytes/q proxy from Table 3’s mem-read column, which counts the bytes the server’s matrix kernel streams locally per scoring call. The field-element schemes (EMVP and BNTM) sit $2.7\text{--}3.4\times$ above PLAINTEXT on the bandwidth axis at the same recall, while SAP’s scale-and-perturb ciphertexts stay f32-sized and match PLAINTEXT byte-for-byte; this is the in-memory cost that predicts the GPU asymmetry in Section 4.2 without requiring a per-scheme GPU build.

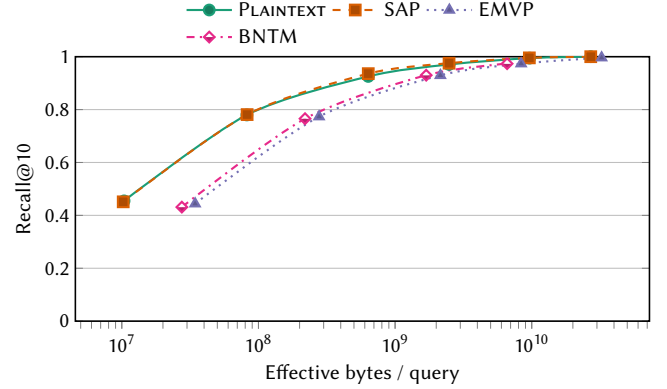


Figure 13: Recall@10 vs. effective-bytes-per-query, the analytical memory-bandwidth proxy that counts the bytes the server’s matrix kernel streams locally per scoring call. Captures the per-query bandwidth premium the field-element matrices impose ($2.7\text{--}3.4\times$ over PLAINTEXT for EMVP and BNTM at the same recall, even though their wire responses differ by orders of magnitude; SAP matches PLAINTEXT) without requiring a per-scheme GPU build. Source: MS MARCO-8.8M.

A.1 Per-scheme parameters

Table 5 consolidates the cryptographic parameters of each scheme, in the source constructions’ own notation, all at the 128-bit security level used throughout (Section 3). Every tuple targets the published 128-bit ($\lambda=128$) setting; the two choices that are ours rather than inherited are EMVP’s zero-padding of the 768-dimensional embeddings up to the record length $\ell_0 = 1024$ and BNTM’s $\lambda' = 3$ Freivalds trials (the budget for 2^{-128} malicious-server detection).

A.2 Derivation of the communication budget

Table 3 is derived from closed-form expressions in the corpus geometry and the per-scheme cryptographic parameters; this subsection states those formulas and lists the parameter values used at each corpus scale. The wire- and memory-byte budget is deterministic once these are fixed, which is what lets us project TIPTOE’s 8.8 M row without an end-to-end run at that scale. The 8.8 M corpus needs $n_{\text{probe}} = 64$ to reach the same recall.

Parameters. Table 6 lists the symbols. The corpus geometry (c, m, m_{max}) is determined by the inverted file partitioning of the corpus; the scheme parameters are fixed by each construction’s security level (a 128-bit level across all schemes — cryptographic-hardness $\lambda=128$ for EMVP, BNTM, and TIPTOE; a 128-bit symmetric key for SAP, whose privacy is the β perturbation rather than a hardness bound; see Section 2.3 and Table 5). The TIPTOE parameters n_{LWE} , the BFV polynomial degree ρ_{BFV} , the per-ciphertext byte cost b_{BFV} , and the limb count L come directly from the SimplePIR / BFV security configuration used in the implementation [10, 15, 16].

Table 5: Per-scheme parameters, in each construction’s own notation, configured at a 128-bit level (Section 3); the basis differs by scheme, per the *Security basis* column. For the cryptographic schemes (EMVP, BNTM, Tiptoe) it is the source paper’s $\lambda=128$ computational-hardness analysis, which we adopt rather than re-derive. SAP/DCPE is the exception: it carries a 128-bit symmetric key (key secrecy), but has no computational-hardness assumption delivering a 128-bit bound — its server-side privacy is the distance-distorting β perturbation, not a $\lambda=128$ guarantee. EMVP’s listed tuple is our instantiation of the paper’s $\lambda=128$ random-block regime (Table 1), not a tabulated row.

Scheme	Assumption	Parameters	Security basis
PLAINTEXT + IVF	none	—	no cryptography
SAP + IVF	symmetric (DCPE)	$\beta \in \{0, 0.5\}$	key secrecy
EMVP + IVF	secret dual codes	$(n, k, s, b, \ell_0) = (1292, 268, 76, 17, 1024)$ (768-dim padded to ℓ_0)	$\lambda=128$ regime, Table 1 [3]
BNTM + IVF	LPN	$(n, n_1, \delta, \epsilon, \mu) = (1024, 128, 0.125, 0.7, n^{-0.3})$, $\lambda' = 3$	$\lambda=128$ target via the paper’s heuristic (uncertified); 2^{-128} detection [7]
Tiptoe	LWE	$(n, \log_2 p, \sigma) = (2048, 17, 81\,920)$; BFV at published params	published params [15]

Table 6: Communication-budget parameters used in Table 3.

Symbol	Meaning	100 k	8.8 M
N	corpus size	100 000	8 800 000
c	cluster count ($\lceil \sqrt{N} \rceil$)	317	2 967
m	mean cluster size (N/c)	315	2 966
$m_{\max}$	largest cluster (Tiptoe padding target)	977	9 186 [‡]
n_{probe}	probed clusters per query at recall@10 ≈ 0.9	32	64
k	top- k results returned	10	10
d	embedding dimension	768	768
F	field byte width (u64) for crypto schemes	8	8
n_{EMVP}	EMVP encoded-row width	1292	1292
s_{EMVP}	EMVP block count per cluster	76	76
n_{BNTM}	BNTM encoded-row width	1024	1024
n_{LWE}	Tiptoe LWE dimension	2048	2048
ρ_{BFV}	Tiptoe BFV polynomial degree	4096	4096
b_{BFV}	per-BFV-ciphertext byte cost	20 480	20 480
L	Tiptoe BFV limb count	8	8

[‡] The 8.8 M value of $m_{\max}$ is projected from the 100 k max-to-mean ratio ($977/315 \approx 3.10$) applied to the 8.8 M mean cluster size; we have not partitioned the 8.8 M corpus through Tiptoe’s encoder. The assumption is that k-means cluster-size dispersion is roughly scale-invariant for the same data distribution.

Per-scheme formulas. Table 7 states the closed-form expression behind each cell in Table 3. Three observations: (i) the online *query* ($\uparrow$) is a function of the scheme’s input encoding only, not of corpus size, except for Tiptoe where the encrypted LWE vector has one slot per (cluster, dimension) pair and therefore grows with c ; (ii) the per-query *response* ($\downarrow$) of the encrypted-score schemes scales with n_{probe} and with the per-cluster row count m (or $m_{\max}$ for Tiptoe, since the SimplePIR answer covers the padded cluster); PLAINTEXT and SAP score in the clear on the server and return only the global top- k (a 4 B identifier and a 4 B score per hit, 80 B at $k = 10$), independent of n_{probe} and corpus size; (iii) the one-time *setup* for the IVF schemes scales linearly with the corpus ($c \cdot m \cdot n \cdot F$), so the 8.8 M block of Table 3 sits 88× above the 100 k block.

Extrapolation to 8.8 M.. The 8.8 M block of Table 3 applies Table 7 at the 8.8 M cluster geometry. All rows except Tiptoe’s are end-to-end measurements from runs at this scale; Tiptoe[†] is computed from the formulas, since the LWE matvec was not run end-to-end at 8.8 M (the LWE element width pushes the end-to-end run out of scope for our hardware budget). The 100 k agreement between formulas and measurements is what justifies trusting the Tiptoe projection here.

Worked examples. Substituting Table 6 into Table 7 reproduces each cell of Table 3. Two representative cases:

- BNTM setup at 8.8 M: $c \cdot m \cdot n_{\text{BNTM}} \cdot F = 2967 \times 2966 \times 1024 \times 8 \approx 72.1$ GB.
- Tiptoe BFV offline $\uparrow$ at either scale: $n_{\text{LWE}} \cdot b_{\text{BFV}} = 2048 \times 20\,480 = 42$ MB. This term has no N dependence — it is the cost of transmitting one BFV ciphertext per LWE secret-vector slot, and the LWE dimension is fixed by the security level.

Analytical vs. realised values. The formulas use the partition-wide mean cluster size $m = N/c$. The per-query realised cost reported by the harness uses the empirical mean over the probed cluster set, which at 8.8 M tracks 10–15 % higher than N/c because (i) k-means does not produce perfectly balanced clusters at this scale and (ii) IVF routing biases the probe set toward denser clusters. The 8.8 M block of Table 3 reports the realised values for all rows except Tiptoe’s; the 100 k block agrees with the formulas within rounding because the cluster-size variance is small at the 100 k validation scale. Setup and Tiptoe’s offline-phase costs are independent of routing and match the formulas exactly at both scales. SAP’s setup entries are computed from the formula ($c \cdot m \cdot Fd$) rather than measured.

Table 7: Closed-form byte formulas behind each column of Table 3. Symbols are defined in Table 6. Let $p = \text{nprobe}$.

Scheme	query $\uparrow$	response $\downarrow$	setup	offline $\uparrow$	offline $\downarrow$	eff-bytes/q
PLAINTEXT + IVF	$4d$	$k \cdot (4 + 4)$	—	—	—	$p \cdot m \cdot 4d$
SAP + IVF	Fd	$k \cdot (4 + 4)$	$c \cdot m \cdot Fd$	—	—	$p \cdot m \cdot 4d$
EMVP + IVF	Fn_{EMVP}	$p \cdot s_{\text{EMVP}} \cdot m \cdot F$	$c \cdot m \cdot n_{\text{EMVP}} \cdot F$	—	—	$p \cdot m \cdot n_{\text{EMVP}} \cdot F$
BNTM + IVF	Fn_{BNTM}	$p \cdot m \cdot F$	$c \cdot m \cdot n_{\text{BNTM}} \cdot F$	—	—	$p \cdot m \cdot n_{\text{BNTM}} \cdot F$
TIPTOE	$c \cdot d \cdot F$	$m_{\text{max}} \cdot F$	$m_{\text{max}} \cdot n_{\text{LWE}} \cdot F$	$n_{\text{LWE}} \cdot b_{\text{BFV}}$	$\lceil m_{\text{max}} / \rho_{\text{BFV}} \rceil \cdot L \cdot b_{\text{BFV}}$	$m_{\text{max}} \cdot n_{\text{LWE}} \cdot F$